

第四章：语法分析

语法分析概述
自顶向下语法分析概述
三个重要的集合

1. 语法分析概述

- ◆ 1.1 语法分析程序的功能
- ◆ 词法分析器按照词法规则将字符串形式的源程序识别为单词(Token)序列, 这些单词便是上下文无关文法中的终极符。
- ◆ 语法分析器的功能是: 把词法分析的输出(Token序列)作为输入, 按照文法规则, 从其中识别出各类语法成分(非终极符), 同时进行语法检查, 为语义分析和代码生成做准备。

1.2 语法错误类别

- ◆ 程序中的错误：词法错误、语法错误、语义错误
- ◆ 常见的语法错误有下面一些：
- ◆ 1)程序的开始(后继)单词错：表达式的开始单词错(id, (, n), 语句的开始单词错(id, if, while), 表达式的后继单词错(,), +, *, ,,), 语句的后继单词错(,,})等；
- ◆ 2)标识符和常量单词错：如域名不是标识符，在int, typedef后面不是标识符，枚举类型中的元素不是标识符，switch语句中的情形分量不是所允许的常量等
- ◆ 3)括号类错误：(-), [-], {-}, if -else不配对
- ◆ 4)分隔符错：赋值语句左部变量后面不是赋值号，标识符表或表达式表的分隔符错

1.3 语法错误处理

- ◆ 语法分析器在分析输入符号的过程中，一旦发现错误，应该作适当的恢复。通常编译器试图恢复到某一个状态，以便可以继续分析输入串，使错误不断地被指出。但如果恢复不当，可能会导致大量伪错误的出现，甚至可能会引入语义的伪错误。
- ◆ 几种错误恢复策略：
 - ◆ 1) **紧急方式恢复**：每次跳过一个输入记号，直至某个指定的同步记号，如 “; ,]” 等
 - ◆ 2) **短语级恢复**：对剩余输入做局部纠正，用可以使分析器继续分析的符号串代替剩余输入串的前缀，如 “用; 代替, ” , “删除多余; ” “插入遗漏的; ” 等
 - ◆ 3) **出错产生式**：扩充语言的文法，增加产生出错结构的产生式
 - ◆ 4) **全局纠正**：寻找代价最小的算法，修正输入错误，尚处于理论研究阶段

1.4 自顶向下语法分析概述

- ◆ 自顶向下分析是从文法的开始符号出发，试图为输入串建立一个最左推导，或者为输入串构造一棵语法树的过程。

上下文无关文法G:

(1) $Z \rightarrow aBd$

(2) $B \rightarrow bB$

(3) $B \rightarrow c$

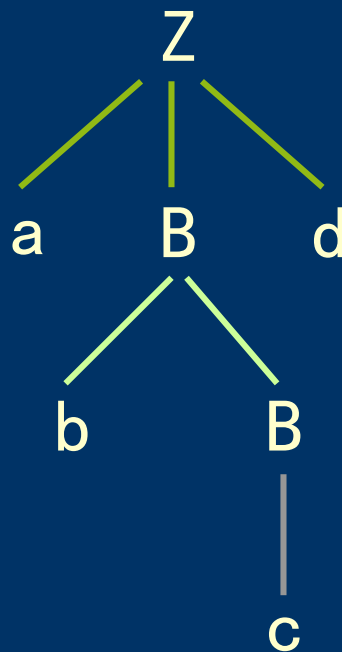
(4) $B \rightarrow d$

$Z \Rightarrow^+ abcd?$

$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} abcd$

$Z \Rightarrow^+ abd?$

$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} abd\mathbf{d}$



1.4 自顶向下语法分析概述

- ◆ 自顶向下分析分为：
- ◆ 非确定的自顶向下分析---推导时随机选择产生式，匹配成功则继续向下推导直至成功，若中间某步匹配不成功则回溯，直至所有可能都尝试过。

$Z \Rightarrow^+ abcd?$	(1) $Z \rightarrow aBd$
$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} abcd$	(2) $B \rightarrow bB$
$Z \Rightarrow^+ abd?$	(3) $B \rightarrow c$
$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} ? ? ?$ 回溯	(4) $B \rightarrow d$
$\Rightarrow_{lm} acd \Rightarrow_{lm} ? ? ?$ 回溯	
$\Rightarrow_{lm} add$ 失败	

- ◆ 确定的自顶向下分析：（被分析文法应满足一定条件，分析时利用某种信息帮助确定性的选择产生式）
 - LL (1) 方法
 - 递归下降方法

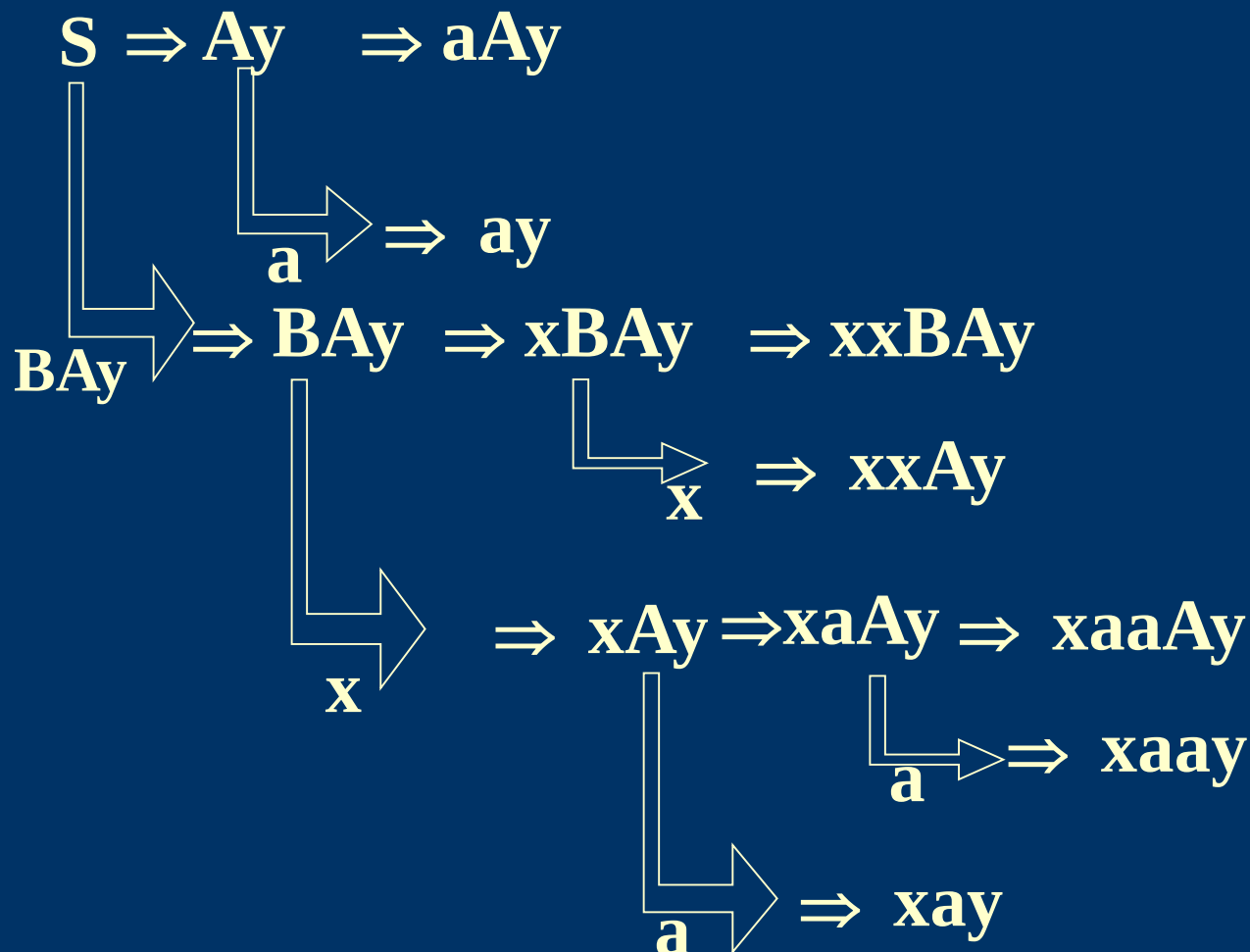
非确定的自顶向下语法分析方法（带回溯过程的自顶向下语法分析方法）

已知 $G[S]$:

$S \rightarrow Ay \mid BAy$

$A \rightarrow aA \mid a$

$B \rightarrow xB \mid x$ 输入串为 xay 是否正确。



确定的自顶向下语法分析

- ◆ 穷举的方法效率非常低，在选择规则时应考虑更多的信息，如输入流。

$Z \Rightarrow^+ abcd?$

$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} abcd$

$Z \Rightarrow^+ abd?$

$Z \Rightarrow_{lm} aBd \Rightarrow_{lm} abBd \Rightarrow_{lm} abdd$

(1) $Z \rightarrow aBd$

(2) $B \rightarrow bB$

(3) $B \rightarrow c$

(4) $B \rightarrow d$

- ❖ 应考察输入流中的几个符号？（通常1个）

2.1 First集的定义

- ◆ 设 $G=(V_T, V_N, S, P)$ 是上下文无关文法,
 $\beta \in (V_T \cup V_N)^*$
$$\text{First}(\beta) = \{ a \in V_T \mid \beta \Rightarrow^* a... \}$$

$$\cup (\text{if } \beta \Rightarrow^* \varepsilon \text{ then } \{\varepsilon\} \text{ else } \emptyset)$$

2.2 Follow集的定义

设 $G=(V_T, V_N, S, P)$ 是上下文无关文法,
 $A \in V_N$, S 是开始符号

$$\text{Follow}(A) = \{ a \in V_T \mid S \Rightarrow^+ \dots Aa \dots \} \\ \cup (\text{if } S \Rightarrow^* \dots A \text{ then } \{\#\} \text{ else } \emptyset)$$

2.3 Predict集的定义

$\text{Predict}(A \rightarrow \beta)$

$= \text{First}(\beta)$, 当 $\text{First}(\beta)$ 不含 ε

$= \text{First}(\beta) - \{\varepsilon\} \cup \text{Follow}(A)$,
当 $\text{First}(\beta)$ 含 ε

3.1 计算First(X)集

- ◆ 若 $X \in V_T$, $\text{First}(X) = \{X\}$
- ◆ 若 $X \in V_N$ 则
$$\text{First}(X) = \{a \mid X \rightarrow a \cdots \in P, a \in V_T\}$$
- ◆ 若 $X \in V_N$, 且有产生式 $X \rightarrow \varepsilon$, 则 $\varepsilon \in \text{First}(X)$
- ◆ 若 $X \in V_N$, 有产生式 $X \rightarrow Y_1 Y_2 \cdots Y_n$, 且 $Y_1, Y_2, \cdots, Y_i \in V_N$
当 $Y_1, Y_2, \cdots, Y_{i-1} \Rightarrow^* \varepsilon$,
则 $\text{First}(Y_1) - \{\varepsilon\}, \text{First}(Y_2) - \{\varepsilon\}, \cdots$
 $\text{First}(Y_{i-1}) - \{\varepsilon\}, \text{First}(Y_i)$ 都包含在 $\text{First}(X)$ 中。
当 $Y_i \Rightarrow^* \varepsilon (i=1, 2, \cdots, n)$,
 $\text{First}(Y_i)$ 都包含在 $\text{First}(X)$ 中。

3.2 计算First(α)集

设符号串 $\alpha = X_1 X_2 \cdots X_n$,
当 $X_1, X_2, \dots, X_{i-1} \Rightarrow^* \varepsilon$, X_i 不能 $\Rightarrow^* \varepsilon$,
则:

$$\text{First}(\alpha) = \bigcup_{j=1}^{i-1} (\text{First}(X_j) - \{\varepsilon\}) \cup \text{First}(X_i)$$

◆ 若所有 X_i 都能 $\Rightarrow^* \varepsilon$,
则:

$$\text{First}(\alpha) = \bigcup_{j=1}^n \text{First}(X_j)$$

文法符号	First集
E	{ id, (}
E'	{ +, ε }
T	{ id, (}
T'	{ *, ε }
F	{ id, (}

例:

- ◆ $E \rightarrow T E'$
- ◆ $E' \rightarrow + T E' \mid \varepsilon$
- ◆ $T \rightarrow F T'$
- ◆ $T' \rightarrow * F T' \mid \varepsilon$
- ◆ $F \rightarrow \text{id} \mid (E)$

符号串	First集
$T E'$	{ id, (}
$F T'$	{ id, (}
$E' T' T$	{ +, *, id, (}
$E' T'$	{ +, *, ε }

4. 计算Follow集

1: 对所有 $A \in V_N$ 且 A 非开始符 :

 令 $\text{Follow}(A) := \{ \}$;

 对开始符 S : 令 $\text{Follow}(S) = \{ \# \}$;

2: 若有产生式 $A \rightarrow xBy$:

 如果 $\varepsilon \in \text{First}(y)$ 则:

$\text{Follow}(B)$

$= \text{Follow}(B) \cup (\text{First}(y) - \{\varepsilon\}) \cup \text{Follow}(A)$

 否则:

$\text{Follow}(B)$

$= \text{Follow}(B) \cup \text{First}(y)$

3: 重复2, 直至对所有 $A \in V_N$, $\text{Follow}(A)$ 收敛为止。

文法符号	First集	Follow集
E	{ id, (}	{ # ,) }
E'	{ +, ε }	{ # ,) }
T	{ id, (}	{ + , # ,) }
T'	{ *, ε }	{ + , # ,) }
F	{ id, (}	{ *, + , # ,) }

例:

- ① $E \rightarrow \underline{T} \underline{E'}$
- ② $E' \rightarrow + \underline{T} \underline{E'}$
- ③ $\quad \quad \quad | \varepsilon$
- ④ $T \rightarrow \underline{F} \underline{T'}$
- ⑤ $T' \rightarrow * \underline{F} \underline{T'}$
- ⑥ $\quad \quad \quad | \varepsilon$
- ⑦ $F \rightarrow id$
- ⑧ $\quad \quad \quad | (E)$

5. 计算Predict集

$\text{Predict}(A \rightarrow \beta)$

$$= \begin{cases} \text{First}(\beta), & \text{当First}(\beta) \text{ 不含 } \varepsilon \\ (\text{First}(\beta) - \{\varepsilon\}) \cup \text{Follow}(A), & \text{当First}(\beta) \text{ 含 } \varepsilon \end{cases}$$

例:

- ◆ $E \rightarrow T E'$
- ◆ $E' \rightarrow + T E' \mid \varepsilon$
- ◆ $T \rightarrow F T'$
- ◆ $T' \rightarrow * F T' \mid \varepsilon$
- ◆ $F \rightarrow \text{id} \mid (E)$

文法符号	First集	Follow集
E	{ id, (}	{), # }
E'	{ +, ε }	{), # }
T	{ id, (}	{ +,), # }
T'	{ *, ε }	{ +,), # }
F	{ id, (}	{ *, +,), # }

$\text{Predict}(E \rightarrow T E') = \text{first}(T E') = \{ \text{id}, (\}$

$\text{Predict}(E' \rightarrow + T E') = \text{first}(+ T E') = \{ + \}$

$\text{Predict}(E' \rightarrow \varepsilon) = \text{follow}(E') = \{), \# \}$

$\text{Predict}(T \rightarrow F T') = \text{first}(F T') = \{ \text{id}, (\}$

例:

- ◆ $E \rightarrow T E'$
- ◆ $E' \rightarrow + T E' \mid \varepsilon$
- ◆ $T \rightarrow F T'$
- ◆ $T' \rightarrow * F T' \mid \varepsilon$
- ◆ $F \rightarrow \text{id} \mid (E)$

文法符号	First集	Follow集
E	{ id, (}	{), # }
E'	{ +, ε }	{), # }
T	{ id, (}	{ +,), # }
T'	{ *, ε }	{ +,), # }
F	{ id, (}	{ *, +,), # }

$$\text{Predict}(T' \rightarrow * F T') = \text{first}(* F T') = \{ * \}$$

$$\text{Predict}(T' \rightarrow \varepsilon) = \text{follow}(T') = \{ +,), \# \}$$

$$\text{Predict}(F \rightarrow \text{id}) = \text{first}(\text{id}) = \{ \text{id} \}$$

$$\text{Predict}(F \rightarrow (E)) = \text{first}((E)) = \{ (\}$$

- ◆ 非确定的自顶向下语法分析方法主要问题：虚假匹配而引起回溯，原因是：

1. 由于相同左部的产生式的右部的FIRST集的交集不为空而引起回溯。

例：有如下文法G[Z]：

$$\text{First}(cAe) \cap \text{First}(cAd) \neq \emptyset$$

$$Z \rightarrow cAe \mid cAd$$

$$A \rightarrow eb \mid a$$

$$\text{Predict}(Z \rightarrow cAe) \cap \text{Predict}(Z \rightarrow cAd) \neq \emptyset$$

分析字符串cad。

■ 至多有一个产生式被选择的条件是：
 $\text{predict}(A \rightarrow \beta_k) \cap \text{predict}(A \rightarrow \beta_j) = \emptyset$ ，当 $k \neq j$

2. 由于某非终极符的产生式的右部能推导出 ϵ ，且该非终极符的FOLLOW集中含有其某个产生式右部的FIRST集中的元素。

$$\text{First}(bAS) \cap \text{First}(\epsilon) = \emptyset$$

例：有如下文法G[S]：

$$\text{First}(bAS) \cap \text{First}(d) = \emptyset$$

$$S \rightarrow aAS \mid b$$

$$\text{First}(d) \cap \text{First}(\epsilon) = \emptyset$$

$$A \rightarrow bAS \mid \epsilon \mid d$$

$$\text{Follow}(A) = \{b, a\}$$

分析字符串ab。

$$\text{Predict}(A \rightarrow bAS) \cap \text{Predict}(A \rightarrow \epsilon) \neq \emptyset$$

■至多有一个产生式被选择的条件是：

$$\text{predict}(A \rightarrow \beta_k) \cap \text{predict}(A \rightarrow \beta_j) = \emptyset, \text{ 当 } k \neq j$$

满足该条件的文法称为LL(1)文法，递归下降子程序文法。

●非LL(1)文法到LL(1)文法的等价转换

- 消除左公共前缀
- 消除直接左递归
- 消除左递归

注意1: 即使文法没有公共前缀和左递归也并不意味着文法一定是LL(1)文法，有时还需要其它的转换方法。

注意2: 通常程序设计语言的文法大都可以转换成LL(1)文法，但已经证明，非LL(1)文法是存在的，即有些文法是无法等价变换成LL(1)文法的。

如下的条件语句文法就不是LL(1)文法：

$\langle \text{Stm} \rangle \rightarrow \text{if } \langle \text{Exp} \rangle \text{ then } \langle \text{Stm} \rangle \text{ else } \langle \text{Stm} \rangle$

$\langle \text{Stm} \rangle \rightarrow \text{if } \langle \text{Exp} \rangle \text{ then } \langle \text{Stm} \rangle$

$\langle \text{Stm} \rangle \rightarrow \dots\dots\dots$

课后练习(1)

$Z \rightarrow aBd$

$B \rightarrow bB$

$B \rightarrow c$

$B \rightarrow d$

First集:

$\text{First}(aBd) = \{a\}$

$\text{First}(bB) = \{b\}$

$\text{First}(c) = \{c\}$

$\text{First}(d) = \{d\}$

Follow集:

$\text{Follow}(Z) = \{\#\}$

$\text{Follow}(B) = \{d\}$

Predict集:

$\text{Predict}(Z \rightarrow aBd) = \{a\}$

$\text{Predict}(B \rightarrow bB) = \{b\}$

$\text{Predict}(B \rightarrow c) = \{c\}$

$\text{Predict}(B \rightarrow d) = \{d\}$

课后练习 (2)

$S \rightarrow ABc$

$A \rightarrow a$

$A \rightarrow \varepsilon$

$B \rightarrow b$

$B \rightarrow \varepsilon$

First集

$\{a, b, c\}$

$\{a\}$

$\{\varepsilon\}$

$\{b\}$

$\{\varepsilon\}$

Follow集

$S = \{\#\}$

$A = \{b, c\}$

$B = \{c\}$

Predict集

$\{a, b, c\}$

$\{a\}$

$\{b, c\}$

$\{b\}$

$\{c\}$